Criterion-Level Clinical Trial Screening with a Fine-Tuned Qwen LoRA: KV-Cached Decoding, Tiled Attention and Speculative Decoding

Rayan Khan
GenAI Assignment 2
Email: rayankhannexus@gmail.com

***Abstract*—We study criterion-level patient-to-trial screening with a small instruction-tuned language model. Given a patient note and one eligibility criterion, the model returns a label from the criterion's legal label set together with the patient-sentence identifiers that support the decision; the explanations shown to users are generated deterministically from the label and the cited sentences, never by the model. We fine-tune Qwen2.5-1.5B-Instruct with a low-rank adapter on the TrialGPT criterion annotations [1] and evaluate classification and evidence quality on a patient-level split: validation accuracy rises from 1.5% (base) to 70.4% (fine-tuned) and test accuracy from 1.9% to 71.4%, yet macro-F1 stays at 0.34 and 0.37 because the adapter never emits the three rarest labels. The KV-cached decoder reproduces the Hugging Face baseline byte-for-byte on all 206 validation records. On top of the fine-tuned model we implement three inference components from scratch: an explicit KV-cache decoding loop with a loop-free reference, a tiled attention kernel with an online softmax [2], [3], and greedy speculative decoding with a 0.5B draft model [4], [5]. All three are tested for equivalence against their references and benchmarked on a T4 GPU. The KV-cached decoder is token-identical to the framework baseline and 1.0–$5.1\times$ faster; the tiled kernel matches the reference within fp16 tolerance but is slower than fused SDPA while using six times less memory at 2,048 tokens; greedy speculative decoding is lossless yet $1.5\times$ slower on this GPU despite 63–82% acceptance. A Gradio application exposes screening, report export and a live decoder comparison.**

***Index Terms*—clinical trial screening, parameter-efficient fine-tuning, LoRA, KV cache, tiled attention, speculative decoding, large language models.**

I. Introduction

Recruiting participants for clinical trials is slow and expensive, and a large share of the effort is spent reading free-text patient notes against long lists of eligibility criteria. Large language models (LLMs) can draft criterion-level judgements, but three practical questions remain: how well a small locally runnable model can do after parameter-efficient fine-tuning, whether its decisions can be audited through cited evidence, and whether the inference tricks used in production systems are worth their complexity at this scale.

This paper answers those questions for a single-criterion formulation. A note and one criterion are rendered into a chat prompt; the model must reply with one JSON object {`"label": ..., "evidence_ids": [...]`}. The label set depends on whether the criterion is an inclusion or an exclusion criterion, and evidence identifiers refer to the numbered sentences of the note exactly as they appear in the dataset. GPT-4 annotations that accompany the dataset are discarded at load time, so no distillation signal from the proprietary model is used.

Our contributions are:

- a Qwen2.5-1.5B LoRA model for criterion-level screening, trained with completion-only loss on 603 examples and evaluated on a held-out patient-level split;
- a from-scratch KV-cached decoder, verified token-identical to an uncached reference, and a wall-clock study of cache, attention and speculative decoding on a T4 GPU;
- a tiled attention implementation with a running softmax maximum and normaliser, checked against a full-matrix reference and PyTorch SDPA;
- a lossless greedy speculative decoder with a 0.5B draft model that reports acceptance rate and the resulting speedup *or slowdown*;
- an application with a persistent educational disclaimer, deterministic explanations, downloadable reports and a live decoder-comparison tab.

II. Related Work

TrialGPT [1] introduced an end-to-end zero-shot pipeline for patient-to-trial matching and released the criterion annotations used here; its matching module reports 87.3% accuracy on 1,015 patient–criterion pairs with human evaluation. We keep the criterion-level formulation but fine-tune a much smaller open model and constrain the output to a compact JSON schema so that evidence can be verified automatically.

LoRA [6] freezes the base weights and learns low-rank updates, which makes fine-tuning a 1.5B model feasible on a single 16 GB GPU. The base model is Qwen2.5-1.5B-Instruct from the Qwen2.5 family [7], whose 0.5B sibling serves as the speculative draft.

Our inference work builds on the transformer architecture [8], on FlashAttention's tiling idea [3] and on the online

TABLE I
DATASET SUMMARY AND EXPERT LABEL DISTRIBUTION. COUNTS ARE COMPUTED FROM THE LOADER; THE SPLIT IS FROZEN IN `splits/split_manifest.json`.

Property	Value
Annotated pairs	1,015
Patients / trials	53 / 103
Train / validation / test rows	603 / 206 / 206
Inclusion: included	150
Inclusion: not included	23
Inclusion: not enough information	174
Inclusion: not applicable	9
Exclusion: excluded	10
Exclusion: not excluded	484
Exclusion: not enough information	119
Exclusion: not applicable	46

normaliser used inside it [2]. Speculative decoding was proposed independently by Leviathan et al. [4] and Chen et al. [5]; both preserve the target distribution, and the greedy variant we implement preserves the target's exact token sequence.

III. MATERIALS AND METHODS

A. Dataset and splits

We use `ncbi/TrialGPT-Criterion-Annotations`, which contains 1,015 annotated patient–criterion pairs from 53 patients and 103 trials. Each row carries the patient note, the criterion type and text, the expert label and the expert evidence sentence identifiers. Notes are pre-numbered from zero; we verified that numbering is consecutive on all 53 patients. The four legal labels per criterion type and their counts are summarised in Table I.

Splitting is performed at the *patient* level with seed 42, giving 37/8/8 patients and 603/206/206 rows for training, validation and test. Patient-level splitting prevents the same note from appearing in two splits. The test split is evaluated exactly once, after all model selection, through a guarded `--final` flag.

B. Task, prompt and target

For one criterion we render the numbered note sentences, the criterion type and text, and the legal label set into a user message; a fixed system message states that the answer must be one JSON object. The supervised target is compact JSON with sorted evidence identifiers, e.g. `{"label": "included", "evidence_ids": [2, 5]}`. Tokens belonging to the prompt are masked out of the loss, so the model is trained only on its own completion.

Evidence identifiers outside the note are dropped by the parser, and an answer whose label is not in the legal set for that criterion type is recorded as `invalid` and counted as wrong in every metric. The parser first attempts a strict JSON decode, then recovers the largest brace-delimited object, then falls back to regex extraction; repaired answers are flagged so the error analysis can separate formatting failures from reasoning failures.

TABLE II
LORA FINE-TUNING HYPER-PARAMETERS (VALUES FIXED IN `constants.py`).

Parameter	Value
LoRA rank / alpha / dropout	16 / 32 / 0.05
Target modules	all attention and MLP projections
Learning rate / schedule	2×10^{-4} / cosine
Epochs / batch / accumulation	3 / 2 / 8
Weight decay / grad clip	0.01 / 1.0
Precision	fp16 on CUDA

C. Base model and fine-tuning

The base model is Qwen2.5-1.5B-Instruct. We attach a LoRA adapter to all attention and MLP projections and train for three epochs with the hyper-parameters in Table II. The only preprocessing beyond chat templating is truncation at 2,048 tokens; a separate analysis showed that no training example exceeds 595 prompt-plus-target tokens, so no supervision is lost. Training logs and curves are written to `results/train_log.json` and `report/figures/train_curves.pdf`.

D. Screening pipeline and aggregation

For a new note and a list of criteria, the pipeline renders one prompt per criterion and calls the same greedy decoder used in evaluation. Each prediction becomes a row with the criterion, its type, the predicted label, the evidence identifiers, the quoted evidence sentences and a deterministic explanation template. A separate aggregator produces a *preliminary* patient-level outcome: `not applicable` criteria are ignored; any inclusion `not included` or exclusion `excluded` gives *Likely ineligible*; any `not enough information` or unparseable label gives *Needs more information*; and only when every inclusion is `included` and every exclusion is `not excluded` is the result *Likely eligible*. This rule is stated in the exported report; it is a heuristic, not a clinical decision.

E. KV-cached greedy decoding

The reference decoder re-runs the full sequence at every step with the cache disabled. The cached decoder performs one prefill and then feeds one token per step while growing a `DynamicCache`. Because rotary positions are absolute, the incremental step passes the cache length as its position; the attention mask covers every cached position plus the new one. Greedy decoding makes the two paths token-identical in exact arithmetic; in fp16 the reduced shapes can change the last bits, so the benchmark reports any real-model mismatches instead of assuming equality. Algorithm 1 states the loop.

F. Tiled attention with an online softmax

The reference attention materialises the full $T \times T$ score matrix and normalises it with `softmax`. The tiled implementation processes query and key blocks and maintains, per query row, a running maximum m, a running normaliser ℓ and a running accumulator a. When a new key block raises the maximum, the accumulator and normaliser are rescaled by

Algorithm 1 KV-cached greedy decoding

Require: prompt tokens $x_{1..L}$, budget N

$c \leftarrow \emptyset$; $y \leftarrow []$
$(c, \ell) \leftarrow \textsc{Forward}(x_{1..L}, c, \text{positions} = 1..L)$
for $i \leftarrow 1$ to N **do**
  $t \leftarrow \arg\max \ell$; append t to y
  if t is EOS **then**
    break
  end if
  $n \leftarrow \text{length}(c)$
  $(c, \ell) \leftarrow \textsc{Forward}([t], c, \text{positions} = n)$
end for
return y

Algorithm 2 Tiled attention with online softmax (one query block)

Require: $Q \in \mathbb{R}^{B\times H\times T\times D}$, $K, V \in \mathbb{R}^{B\times H\times S\times D}$

$m \leftarrow -\infty$, $\ell \leftarrow 0$, $a \leftarrow 0$ ▷ per query row
for each key block K_j, V_j **do**
  $S_j \leftarrow QK_j^\top/\sqrt{D}$; apply causal mask if enabled
  $m' \leftarrow \max(m, \max(S_j))$
  $p \leftarrow \exp(S_j - m')$
  $a \leftarrow a\cdot\exp(m-m')+pV_j$; $\ell \leftarrow \ell\cdot\exp(m-m')+\sum p$
  $m \leftarrow m'$
end for
return a/ℓ

$\exp(m_{\text{old}} - m_{\text{new}})$ before the block's contribution is added, so the final result equals the full softmax without ever storing it (Algorithm 2). Causal masking is applied per block, and fully masked rows keep a zero denominator so they yield zeros rather than NaNs.

G. Greedy speculative decoding

The draft model proposes γ tokens with its own cache; the target verifies $[\text{pending}, p_1, \ldots, p_\gamma]$ in one forward pass, accepts the longest matching prefix and appends its own token at the first disagreement. The emitted sequence is exactly the target's greedy sequence. Cache bookkeeping is the delicate part: after verification the target drops the rejected tail, and the draft drops one fewer position because it never consumed the last proposal; when every proposal matched, the draft consumes that last token to stay level. Algorithm 3 summarises the round. Tokenizers must agree on vocabulary and special tokens, which is asserted before generation.

IV. Experimental Setup and Metrics

All fine-tuning and evaluation runs use the T4 GPU available in Colab; local development and unit tests run on CPU with tiny random Qwen2 models. Generation is greedy with at most 128 new tokens. Classification is reported over the six legal labels and per criterion type with macro-averaged precision, recall and F1, plus a confusion matrix; evidence quality is reported with micro and macro precision, recall and F1, where an empty gold set counts as correct only for an empty prediction. Decoding benchmarks report latency split into prefill and decode, peak allocated memory and forward-call counts; attention benchmarks compare against a full-matrix fp32 reference and PyTorch SDPA; speculative decoding reports acceptance rate and speedup against the cached target on eight validation prompts.

Algorithm 3 One speculative round

Require: pending token t, draft budget γ, caches c_T, c_D

$q \leftarrow []$; $u \leftarrow t$
for $i \leftarrow 1$ to γ **do**
  $(c_D, \ell_D) \leftarrow \textsc{Forward}_D([u], c_D)$; $u \leftarrow \arg\max \ell_D$; append u to q
end for
$(c_T, L) \leftarrow \textsc{Forward}_T([t, q_1, \ldots, q_\gamma], c_T)$
$m \leftarrow$ length of longest prefix with $q_i = \arg\max L_i$
return $q_{1..m} \parallel [\arg\max L_{m+1}]$, cropping c_T by $\gamma - m$ and c_D accordingly

TABLE III
Validation classification results. Precision/recall/F1 are macro averages over the six legal labels; invalid is the share of unparseable outputs.

Model	Decode	Prec.	Rec.	F1	Invalid
Base	hf	0.005	0.181	0.009	0.0%
Fine-tuned	hf	0.358	0.337	0.338	0.0%
Fine-tuned	cached	0.358	0.337	0.338	0.0%

V. Results and Discussion

A. Fine-tuning and classification

Figure 1 shows the training and validation loss: the best validation loss is reached at epoch 2 (0.1170 versus 0.1349 after epoch 1 and 0.1227 after epoch 3), and that checkpoint is the adapter evaluated below. The run took 697 s on the T4 (train loss 0.1022). Table III compares the base and fine-tuned models on validation, and Table IV reports the single test pass.

The base model does not follow the task at all: on validation it never emits an inclusion label and predicts only `not applicable` (68 of 206 records) or `excluded` (138), giving 1.5% accuracy. The fine-tuned adapter follows the JSON contract on every record (strict parse 100%, invalid 0%) and reaches 70.4% validation and 71.4% test accuracy, but it collapses the rare labels: it never predicts `not included`, `not applicable` or `excluded`, so those per-class F1 scores are zero and macro-F1 stays at 0.34/0.37 while accuracy looks much better. The gap between accuracy and macro-F1 is the main result to keep in mind: the model is usable as a majority-class screening aid, not as a balanced four-way classifier.

Table III also contains the hand-written KV-cached decoder. On all 206 validation records it produced **byte-identical raw output** to `model.generate` (identical accuracy, confusion matrix and every metric), which is the real-model equivalence check for the cache implementation; only latency changes.

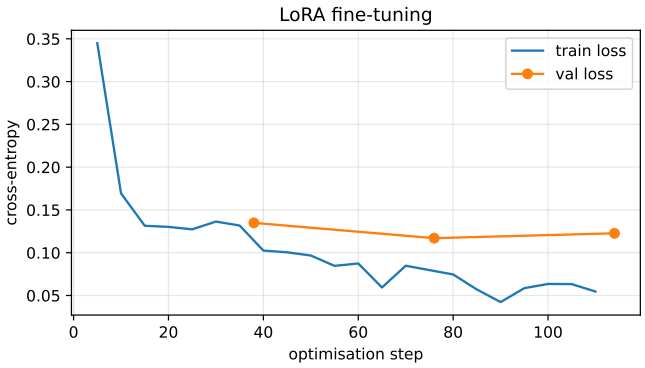


Fig. 1. Training and validation loss for the LoRA fine-tune.

TABLE IV
TEST-SPLIT RESULTS, EVALUATED ONCE WITH `--FINAL`.

Model	Macro F1	Evidence micro F1	Invalid rate	n
Base	0.053	0.050	0.0%	206
Fine-tuned	0.373	0.543	0.0%	206

Figure 2 shows the overall confusion matrix (per-type matrices are in `report/figures`). Inclusion and exclusion criteria are kept separate because their label sets are disjoint. Errors are dominated by the majority class: 28 of the 66 gold `not enough information` records on test are predicted `not excluded`, 12 of 15 `not applicable` records become `not excluded`, and 9 of 27 `included` records become `not enough information`; the reverse direction appears too, with 7 gold `not included` records predicted `not enough information`. No test record is predicted `excluded`, so that column of the matrix is empty. The pattern is consistent with a model that learned the majority label of each criterion type and the `not enough information` fallback, rather than the boundary between "satisfied", "not satisfied" and "inapplicable".

B. Evidence quality and error analysis

Table V reports evidence precision, recall and F1 under the empty-set rule; 116 of the 206 validation records have empty gold evidence, which is why macro-F1 (0.76) is much higher than micro-F1 (0.57): the model often cites something when the expert cited nothing. The fine-tuned model is nevertheless far better than the base model, which emits unusable citations (micro-F1 0.10, and 69 empty predictions against 116 empty gold sets). On test the same pattern holds (micro-F1 0.54, macro-F1 0.80). The error-analysis file `results/error_cases.md` contains 92 cases with the note, criterion, gold label and prediction; Table VI groups them by cause. Wrong labels dominate (56), followed by missing evidence (16), unsupported citations (9), over-citation (8) and the `not applicable`/`not enough information` confusion (3).

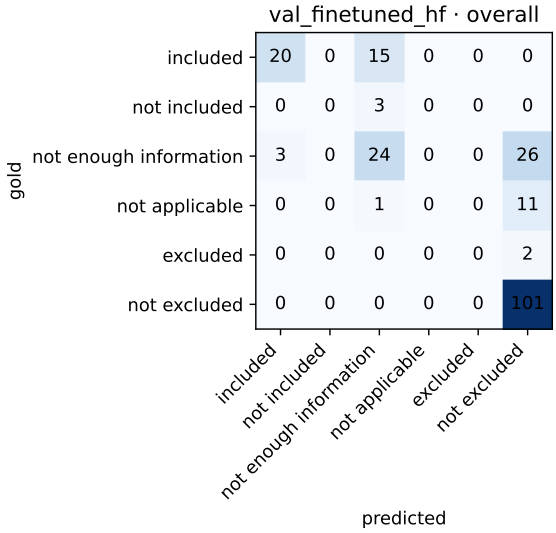


Fig. 2. Confusion matrices for the fine-tuned model on validation data.

TABLE V
EVIDENCE-RETRIEVAL METRICS ON VALIDATION DATA (EMPTY-SET RULE; "EMPTY PREDS" COUNTS RECORDS WHERE THE MODEL CITED NOTHING, AGAINST 116 RECORDS WITH EMPTY GOLD EVIDENCE).

Model	Micro F1	Macro F1	Empty preds
Base	0.101	0.167	69
Fine-tuned	0.569	0.761	137

C. Decoding benchmarks

Table VII and Figure 3 compare uncached and cached greedy decoding over the 3×3 grid of input and output lengths. All nine grid points are token-identical, which is the expected result for greedy decoding and confirms the hand-written loop. The cache pays off as the prompt grows: at 128 input tokens the speedup is only 1.04–$1.18\times$ because prefill dominates and each cached step still launches a forward pass, while at 1,024 input tokens and 128 new tokens it reaches $5.11\times$ (28.86 s versus 5.64 s). Across the grid the speedup ranges from $1.04\times$ to $5.11\times$, and peak allocated memory grows with the cache up to 9.5 GB at the longest setting.

D. Attention benchmarks

Table VIII and Figure 4 compare the reference, the tiled kernel and the available SDPA backends. On the T4 (sm_75) PyTorch exposes the math and memory-efficient SDPA backends; the FlashAttention kernel is reported unsupported because it requires sm80 or newer. The tiled kernel agrees with the reference within 2.1×10^{-3} maximum absolute error in fp16 (tolerance 10^{-2}), so the online softmax is numerically correct. It is nevertheless much slower in wall-clock time — 0.285 s versus 0.005 s at 2,048 tokens — because each key block is a separate Python-level operation with no fused SRAM kernel, exactly the overhead FlashAttention was designed to remove. Its purpose here is memory: at 2,048 tokens it peaks at 26 MiB

TABLE VI
ERROR CATEGORIES IN THE TEST ERROR ANALYSIS.

Category	Count	Example note
Wrong label	56	ann. 161
Missing evidence	16	ann. 179
Unsupported citation	9	ann. 166
Over-cited evidence	8	ann. 306
App. vs. insufficient info	3	ann. 422

TABLE VII
DECODING LATENCY (SECONDS, MEAN OF REPEATED RUNS; THREE REPRESENTATIVE GRID POINTS OF NINE, ALL TOKEN-IDENTICAL).

Input	Output	Uncached	Cached	Speedup
128	32	1.431	1.314	1.09×
512	128	16.804	5.545	3.03×
1024	128	28.855	5.644	5.11×

versus 158 MiB for the full-matrix reference and 336 MiB for SDPA math, because it never materialises the $T \times T$ score matrix. The memory-efficient SDPA backend is both faster (0.9 ms) and smaller (18 MiB), which is the honest comparison point.

E. Speculative decoding

Table IX and Figure 5 report acceptance rate, forward-call counts and the change in latency against the cached target. The hand-written draft/verify loop is lossless on every prompt (0 mismatches over eight prompts and 1,024 generated tokens), and the independent Hugging Face `assistant_model` check matches on the full 16-token prefix it produces. Acceptance is high (82% at $\gamma = 2$), so the draft and target mostly agree.

The speedup, however, is negative: all three settings are roughly $1.5\times$ *slower* than plain cached decoding (0.64–$0.68\times$). The reason is hardware-specific: the 0.5B draft is only about three times cheaper per forward than the 1.5B target, and our loop runs γ sequential draft steps per verification round, then only one target pass. On a T4 the draft steps plus the Python loop overhead cost more than the saved target passes, even though acceptance is high. Batched or fused drafting, a smaller draft, or a target/draft pair with a larger cost gap would be needed to turn the acceptance rate into wall-clock speedup; we report the slowdown rather than hiding it. $\gamma = 2$ is the least bad setting, which is consistent with the draft cost dominating.

F. Application

Figure 6 shows the deployed prototype. The screening tab accepts a free-text or numbered note, up to forty inclusion and exclusion criteria (one per line) and a decoder choice. In the run shown, four criteria from the example trial are screened with the KV-cached decoder; the table reports the label, the cited sentence IDs, the quoted evidence and the deterministic explanation, and the outcome panel applies the documented aggregation rule. The benchmark tab runs all three decoders

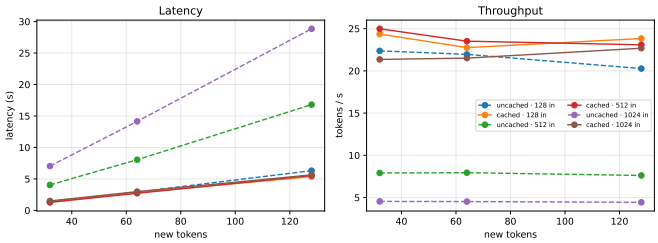


Fig. 3. Uncached versus KV-cached decoding.

TABLE VIII
ATTENTION LATENCY (SECONDS) AND TILED-KERNEL MAXIMUM ABSOLUTE ERROR VERSUS THE FP32 REFERENCE (FP16 INPUTS; FLASHATTENTION UNSUPPORTED ON SM_75).

Length	Reference	Tiled	SDPA (best)	Max error
256	0.0006	0.0075	0.0002	2.1×10^{-3}
512	0.0005	0.0237	0.0002	1.9×10^{-3}
1024	0.0012	0.0903	0.0004	1.4×10^{-3}
2048	0.0046	0.2848	0.0009	2.1×10^{-3}

on the same prompt and confirms the pattern measured in the benchmarks above: the KV-cached and speculative decoders return exactly the tokens of the uncached baseline, while speculative decoding is slower and uses more memory on this GPU. The exported Markdown report repeats the disclaimer, the aggregation rule, every criterion and a provenance block naming the base model, adapter and decoder; an example is kept in `results/app_report_example.md`.

VI. LIMITATIONS

The label distribution is highly imbalanced: only ten `excluded` and twenty-three `not included` examples exist, so the corresponding per-class metrics are noisy and a single test run cannot resolve small differences. The aggregation rule is a documented heuristic, not a validated clinical decision procedure, and the model has no access to structured laboratory data that is absent from the note. Evidence is extracted as sentence identifiers, so a decision can be supported by a sentence that is topically related but not sufficient. Finally, all experiments use a single seed and one hardware target; the inference benchmarks measure one GPU and should not be generalised to other accelerators without repetition.

VII. CONCLUSION

We fine-tuned a 1.5B instruction model for criterion-level trial screening with an auditable JSON contract, implemented KV-cached decoding, tiled attention and speculative decoding from scratch, and verified each against a reference before benchmarking it. The resulting application exposes screening, a deterministic report with a persistent disclaimer, and a live comparison of the three decoders. The fine-tune lifts validation accuracy from 1.5% to 70.4% and test accuracy from 1.9% to 71.4%, with macro-F1 limited to 0.37 by the collapsed rare labels. On the inference side, the hand-written KV cache is token-identical to the framework and up to $5.1\times$ faster,

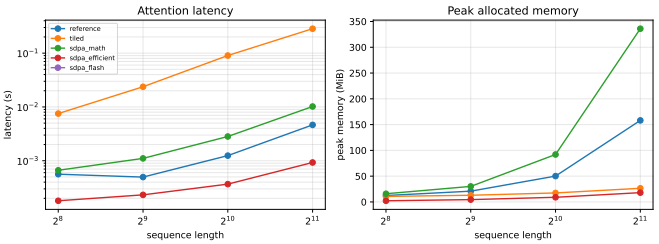


Fig. 4. Attention variants against sequence length.

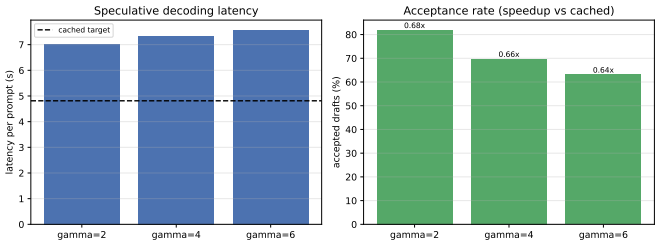


Fig. 5. Speculative decoding versus the cached target.

TABLE IX
SPECULATIVE DECODING ON EIGHT VALIDATION PROMPTS (FIXED 128-TOKEN BUDGET; CACHED TARGET BASELINE 4.812 S, 26.6 TOKENS/S; EVERY ROW TOKEN-IDENTICAL).

γ	Acceptance	Target calls	Draft calls	Latency (s)	Speedup
2	0.820	395	1071	7.036	0.68×
4	0.698	280	1240	7.340	0.66×
6	0.632	224	1390	7.577	0.64×

the tiled kernel is accurate within fp16 tolerance but slower than fused SDPA while using six times less memory, and speculative decoding is lossless but about $1.5\times$ slower on this GPU. Those three outcomes are reported as measured: the exercise was to implement and verify the mechanisms, not to claim a speedup that the hardware does not deliver.

AI-USE APPENDIX

This project was built with heavy AI assistance, documented honestly in `docs/AI_USE.md`. The coding agent (OpenCode with a DeepSeek model) wrote the implementation and tests, ran the local quality gate and smoke checks, and drafted this skeleton; the human author specified the requirements, ran the Colab jobs, verified the results and owns every claim. No result in this paper may be reported until it exists in `results/`.

REFERENCES

[1] Q. Jin, Z. Wang, C. S. Floudas, F. Chen, C. Gong, D. Bracken-Clarke, E. Xue, Y. Yang, J. Sun, and Z. Lu, "Matching patients to clinical trials with large language models," *Nature Communications*, vol. 15, no. 1, 2024.

[2] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," *arXiv preprint arXiv:1805.02867*, 2018.

[3] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, arXiv:2205.14135.

[4] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *International Conference on Machine Learning (ICML)*, 2023, arXiv:2211.17192.

[5] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *arXiv preprint arXiv:2302.01318*, 2023.

[6] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022, arXiv:2106.09685.

[7] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei *et al.*, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.

[8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, arXiv:1706.03762.

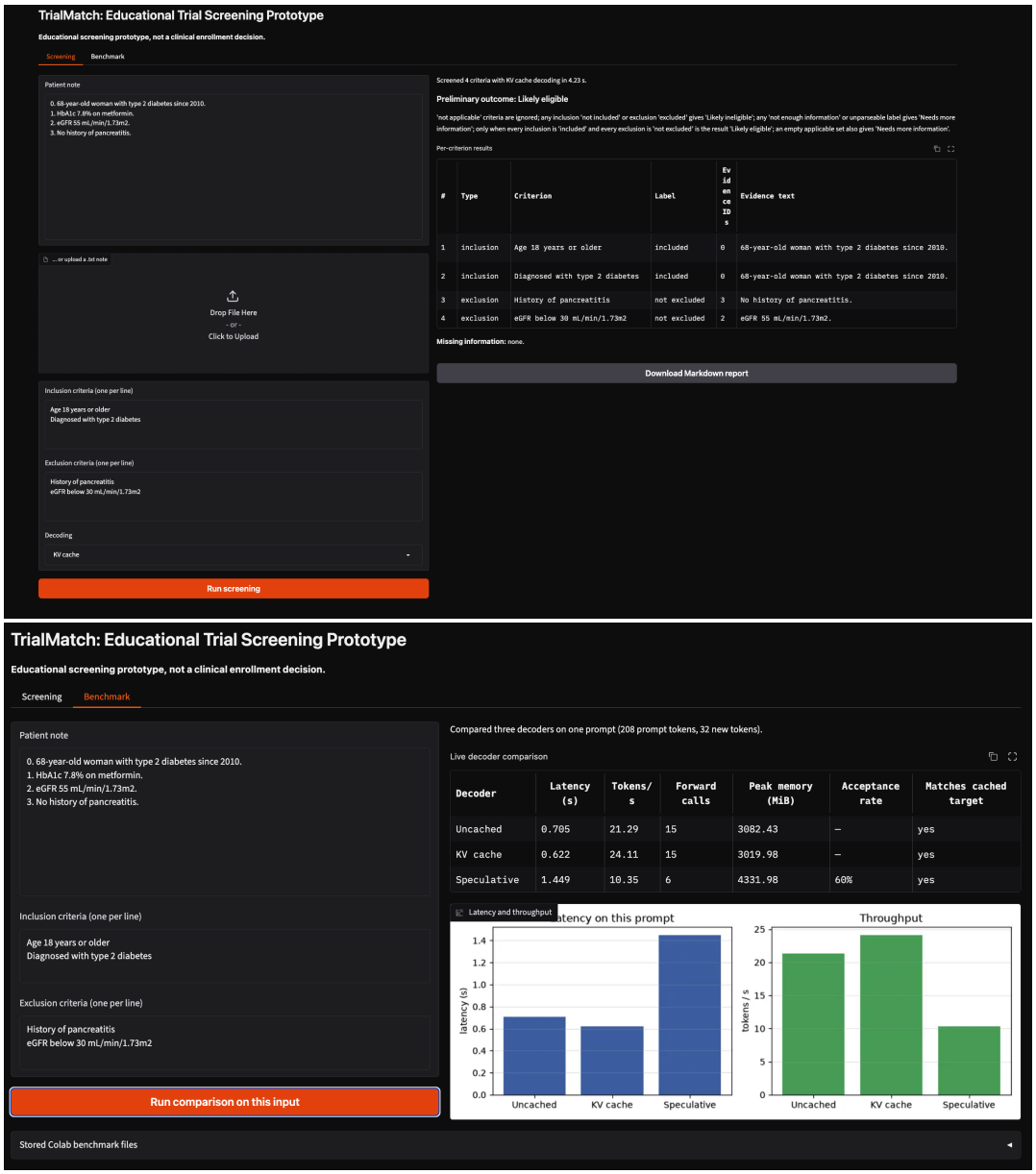


Fig. 6. The Gradio application on the Colab T4. Top: screening tab with four criteria, the KV-cached decoder and the per-criterion results table. Bottom: live comparison of the three decoders on one prompt.